



DOCAPOSTE



Développement de Modelio avec Git

CI/CD Docaposte

MISE À JOUR : JUILLET 2025

Problématiques

Développement de Fonctionnalité

Développement d'une nouvelle fonctionnalité de taille significative.

- Travail collaboratif indépendants.
- Merge avec la branche de développement quant la fonctionnalité est suffisamment avancée pour passer en tests.

Correction de Bugs

Correction d'anomalies sur la branche principale en fin de cycle de développement.

Validation

Validation de la version en cours de développement

- Packaging journalier de la version de développement pour tests.
- Validation pré-pelease
- Validaiton pré-patche

Release

Préparer et packager une version de Modelio.

- Packaging Release Candidate pour la validation
- Packaging de la release courante.
- Packaging de release de version plus ancienne de Modelio.

Patche

Développement de patches correctifs mensuel pour la version courante (dernière release en date)

- Développement cumulatif à partir de la dernière release.
- Tests des Patches correctifs avant livraison
- Packaging de Version Patché
- Report des Patches dans la version en cours de développement

Repository Git Modelio

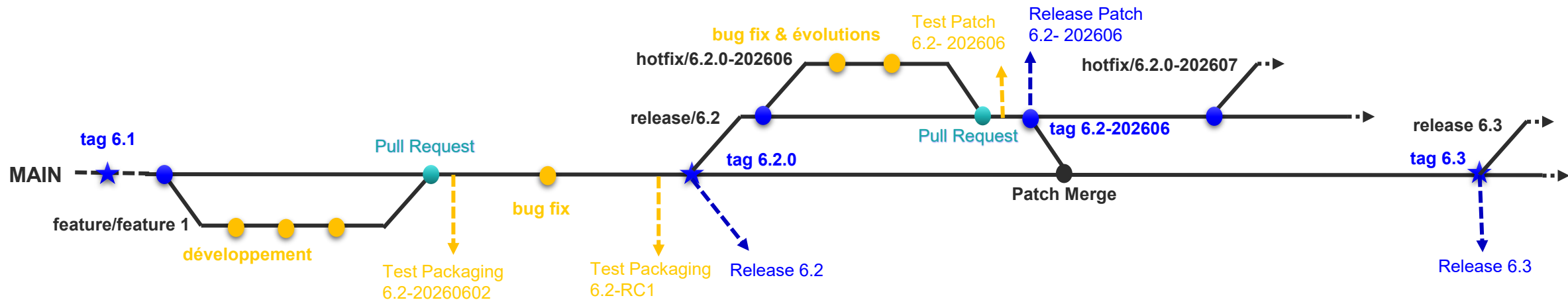
Branches

Branches principales *main* : développement et release de la version

Branches features *feature/{feature name}* : développement des features

Branche par Release *release/{no release}* : archive des releases

Branche par Patches *hotfix/{no patche}* : développement des patches sur branche de maintenance



Développement de Fonctionnalité

WORKFLOW

- ☐ Création d'une branche dédiée à la fonctionnalité en début de développement
- ☐ Commit et collaboration sur la Branche.
- ☐ Quand le développement de la fonctionnalité est suffisamment avancé et que des tests d'intégrations sont nécessaire, on :
 - ☐ Met à jour la branche feature avec les changements de la branche main et on valide avant
 - ☐ Création d'une Pull Request dans GitLab
- ☐ Lorsque la fonctionnalité est complète fonctionnellement, on clôture la branche après une dernière Pull Request

Créer la branche en local et basculer

```
git switch -c feature/ma-fonctionnalite
```

Pousser sur le server et définir la branche distante par défaut

```
git push --set-upstream origin feature/ma-fonctionnalite
```

Commit sur la Branche de la fonctionnalité

Basculer sur la branche de la fonctionnalité et mise a jours /ma-fonctionnalite

```
git pull --rebase
```

Synchroniser en conservant un historique linéaire local

```
git add .
```

```
git commit -m "..."
```

```
git push
```

Mettre à jour la branche de feature régulièrement

```
git fetch origin
```

```
git rebase origin/main
```

Livrer un feature sur la branche main

S'assurer que main est à jour localement puis faire le merge

```
git switch feature/x
```

```
git fetch origin
```

```
git rebase origin/main
```

Régler les conflits et valider l'intégration

Pousser la branche (forcé car le rebase a réécrit l'historique)

```
git push --force-with-lease origin feature/x
```

Créer la Pull Request

Correction d'Anomalies

WORKFLOW

- ❑ En fin de cycle de développement de version, correction des anomalies directement sur la branche main.
 - ❑ Basculer vers la branche principale,
 - ❑ Récupérer les modifications éventuelles de la branche principale et tester que la livraison est toujours valide.
 - ❑ Livrer les modifications locales sur la branche principale.

basculer sur la branche principale

```
git switch main  
git pull --rebase
```

Récupérer les modifications de la branche main avant une livraison

```
git pull --rebase
```

Synchroniser en conservant un historique linéaire local

```
git add .  
git commit -m "..."  
git push
```

RELEASE Modelio

WORKFLOW

- ☐ Création d'un Tag sur la branche principal,
- ☐ Création d'une branche de maintenance pour la version.

Création de Tag

```
git tag 6.2.0 -m « Release Modelio 6.2.0 »  
git push origin 6.2.0
```

Création de Branche de maintenance

```
git switch -c /release/6.2.0
```

Développement de PATCHES

WORKFLOW

- ☐ Basculer sur la branche de maintenance de la version)
- ☐ Création d'une branche dédiée au développement du patche (hotfix/X.X.X-YYYYMM)
- ☐ Livrer les corrections sur la branche de développement du patche.
- ☐ Quand le patche est validé, Création d'une Pull Request dans GitLab
- ☐ Lorsque le Patche est validé :

Basculer sur la branche de maintenance

Créer la branche en local à partir de la branche release

```
git switch release/6.2.0
```

```
git switch -c hotfix/6.2.0-202606
```

Pousser sur le server et définir la branche distante par défaut

```
git push --set-upstream origin hotfix/ 6.2.0-202606
```

Livrer un fix sur la branche de patche

```
git switch hotfix/6.2.0-202606
```

```
git pull
```

```
git add .
```

```
git commit -m "..."
```

```
git push
```

Livrer un Patch sur la branche de maintenance

S'assurer que main est à jour localement puis faire le merge

```
git switch hotfix/6.2.0-202606
```

```
git pull
```

```
git rebase origin/main
```

Régler les conflits et valider l'intégration

Pousser la branche (forcé car le rebase a réécrit l'historique)

```
git push --force-with-lease origin hotfix/6.2.0-202606
```

Créer la Pull Request (on ne compacte pas l'historique des commits)

RELEASE Patch

WORKFLOW

- ☐ Création d'un Tag sur la branche de maintenance (X.X.X-YYYYMM)
- ☐ Intégration du Patch à partir du tag
- ☐ Report du contenu de la branche de maintenance sur la branche main

Création de Tag

```
git switch release/6.2.0
```

```
git tag 6.2.0-202607 -m « Patche Modelio 6.2.0 - 202607 »
```

```
git push origin 6.2.0-202607
```

Reporter le Patch sur main

S'assurer que main est à jour localement

```
git switch main
```

```
git pull origin main
```

Fusionner la branche de maintenance dans main si nécessaire

S'assurer que la fusion n'entraîne pas de conflits et vérifier que tout est toujours ok

```
git merge feature/x
```

```
git push origin main
```

Résolution de conflits

Editer le fichier en conflit et traiter les balises

```
<<<<<< HEAD (votre branche)
ton code actuel
=====
le code venant de la branche d'origine
>>>>>> origin/main
```

Commande	Action
git status	Identifier les fichiers en conflit
git add <fichier-en-conflit>	Marquer le conflit comme résolu
git rebase --continue	Continuer l'opération en cours si tu faisais un rebase
git commit	Continuer l'opération en cours si tu faisais un commit

Intégration

Packaging de Modelio

- ☐ Validation et Release Modelio
 - ☐ Packaging journalier de la branche main
 - ☐ Packaging manuel d'une release a partir de la branche main et de son Tag de Release
- ☐ Validation et Release Patche
 - ☐ Packaging manuel a partir de la branche hotfix
 - ☐ Packaging manuel sur branche de maintenance et tag de patche
- ☐ Packaging d'un version Historique a partie de sa branche de maintenance